

(Property-based testing + зав.типы) × деривация = 🧯

Часть 2. Генераторы

Денис Буздалов

1 июля 2026



Property-based testing

- Тестирование функции/системы на *произвольном* интересном входе
- Оценка, а не предугадывание результата
- Рандомизированная генерация входных значений
- Десятки библиотек под множество языков

Haskell, Erlang, Scala, Python, Coq, Idris, C#, C++, Clojure, D, Elixir, Elm, F#, Go, Java, JavaScript, Julia, Kotlin, Nim, OCaml, Prolog, Racket, Ruby, Rust, Swift, TypeScript, ...

Хорошо применённый property-based testing

- ✓ одна спецификация находит ошибки в разных местах
- ✓ высокоуровневая спецификация может находить низкоуровневые проблемы
- ✓ может найти практически ненаходимое ручным тестированием
- ✓ находит то, о чём не подозревали, *хороший* метод
- ✓ сложность тестирования растёт медленнее сложности SUT
- ✗ надо уметь выбирать подходящие свойства
- ✗ reimplementation trap
- ✗ формализация требований, нужен опыт и mindsetting
 - ✓ инварианты, модели, метаморфное тестирование, автоматы
- ✗ написание генераторов, корректность, полнота, распределения
 - ✓ деривация

Хорошо применённый property-based testing

- ✓ одна спецификация находит ошибки в разных местах
- ✓ высокоуровневая спецификация может находить низкоуровневые проблемы
- ✓ может найти практически ненаходимое ручным тестированием
- ✓ находит то, о чём не подозревали, *хороший* метод
- ✓ сложность тестирования растёт медленнее сложности SUT
- ✗ надо уметь выбирать подходящие свойства
- ✗ reimplementation trap
- ✗ формализация требований, нужен опыт и mindsetting
 - ✓ инварианты, модели, метаморфное тестирование, автоматы
- ✗ написание генераторов, корректность, полнота, распределения
 - ✓ деривация, **type-driven property-based testing**

Dependent types-driven property-based testing

Dependent types-driven property-based testing

— Декларативная спецификация (не обязательно полная!)

```
data SortedNL : Type
```

Dependent types-driven property-based testing

— Декларативная спецификация (не обязательно полная!)

```
data SortedNL : Type
```

— Сдери́вированный полный генератор + хорошие распределения

```
arbitrarySortedNL : Fuel → Gen MaybeEmpty SortedNL
```

```
arbitrarySortedNL = deriveGen
```

Dependent types-driven property-based testing

— Декларативная спецификация (не обязательно полная!)

```
data SortedNL : Type
```

— Сдериивированный полный генератор + хорошие распределения

```
arbitrarySortedNL : Fuel → Gen MaybeEmpty SortedNL
```

```
arbitrarySortedNL = deriveGen
```

— Преобразователь в целевые входные данные

```
toList : SortedNL → List Nat
```


Dependent types-driven property-based testing

— Декларативная спецификация (не обязательно полная!)

```
data SortedNL : Type
```

— Сдериивированный полный генератор + хорошие распределения

```
arbitrarySortedNL : Fuel → Gen MaybeEmpty SortedNL
```

```
arbitrarySortedNL = deriveGen
```

— Преобразователь в целевые входные данные

```
toList : SortedNL → List Nat
```

— Тестируемое свойство (не обязательно полное!)

```
insertOk : Property
```

```
insertOk = property $ \f1 ⇒ do
```

```
  x ← forAll arbitraryNat
```

```
  xs ← forAll $ toList <$> arbitrarySortedNL f1
```

```
  assert $ sorted $ insert x xs
```

```
  assert $ x `elem` insert x xs
```

В прошлых сериях
○○●

Сигнатура генератора
○○○○○

Внутренность: обзор
○○○○

Внутренность: DepTyCheck
○○○

Заключение
○○

Успешный опыт применения

Успешный опыт применения

Один компилятор

9 + 8 багов

ИСП РАН

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия
	SystemVerilog	71

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия
	SystemVerilog	71 (7 с прошлого доклада!)

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия
	SystemVerilog	71 (7 с прошлого доклада!)
	eBPF, EventB	тестировано

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия
	SystemVerilog	71 (7 с прошлого доклада!)
	eBPF, EventB	тестировано
GenProof	Scala	8 падений

Успешный опыт применения

ИСП РАН	<i>Один компилятор</i>	9 + 8 багов
	Scastie (Scala)	1 бага
	FAT32	3 несоответствия
	SystemVerilog	71 (7 с прошлого доклада!)
	eBPF, EventB	тестировано
GenProof	Scala	8 падений
	SystemVerilog	3 падений/зависаний + 6 старых

Что есть генератор?

Что есть генератор?

```
anyBool : Gen0 Bool  
anyBool = oneOf [ pure False, pure True ]
```

Что есть генератор?

```
anyBool : Gen0 Bool  
anyBool = oneOf [ pure False, pure True ]
```

```
data List : Type → Type where  
  Nil  : List a  
  (::)  : a → List a → List a
```

Что есть генератор?

```
anyBool : Gen0 Bool
anyBool = oneOf [ pure False, pure True ]
```

```
data List : Type → Type where
  Nil  : List a
  (::)  : a → List a → List a
empty : List a
empty = []
```

Что есть генератор?

```
anyBool : Gen0 Bool  
anyBool = oneOf [ pure False, pure True ]
```

```
data List : Type → Type where  
  Nil  : List a  
  (::)  : a → List a → List a  
  
empty : List a  
empty = []  
  
oneToFive : List Nat  
oneToFive = [1, 2, 3, 4, 5]
```

Что есть генератор?

```
anyBool : Gen0 Bool
anyBool = oneOf [ pure False, pure True ]
```

```
data List : Type → Type where
  Nil  : List a
  (::)  : a → List a → List a

empty : List a
empty = []

oneToFive : List Nat
oneToFive = [1, 2, 3, 4, 5]

oneToFive' : List Nat
oneToFive' = 1 :: 2 :: 3 :: [4, 5]
```


Что есть генератор?

```
anyList : Gen0 a → Gen0 (List a)
```


Что есть генератор?

```
covering  
anyList : Gen0 a → Gen0 (List a)  
anyList genA = oneOf [ pure []  
                      , do x ← genA  
                          xs ← anyList genA  
                          pure (x::xs)  
                      ]
```

Что есть генератор?

```
anyList : Fuel → Gen0 a → Gen0 (List a)
```

```
covering
```

```
anyList : Gen0 a → Gen0 (List a)
```

```
anyList genA = oneOf [ pure []  
                      , do x ← genA  
                          xs ← anyList genA  
                          pure (x::xs)  
                      ]
```


Что есть генератор?

```
anyList : Fuel → Gen0 a → Gen0 (List a)
anyList Dry _ = pure []
anyList (More f1) genA = oneOf [ pure []
                                , do x ← genA
                                   xs ← anyList f1 genA
                                   pure (x::xs)
                                ]
```

[illegible]

Что есть генератор?

`anyList` : `Fuel` $\rightarrow$ `Gen0` `a` $\rightarrow$ `Gen0` (`List` `a`)

Что есть генератор?

`anyList` : `Fuel` $\rightarrow$ (`Fuel` $\rightarrow$ `Gen0 a`) $\rightarrow$ `Gen0 (List a)`

Что есть генератор?

```
anyList : Fuel → (Fuel → Gen0 a) → Gen0 (List a)
anyList Dry _ = pure []
```

Что есть генератор?

```
anyList : Fuel → (Fuel → Gen0 a) → Gen0 (List a)
anyList Dry _ = pure []
anyList fl'@(More fl) genA = oneOf [ pure []
                                     , do x ← genA fl'
                                       xs ← anyList fl genA
                                       pure (x::xs)
                                     ]
```

Что есть генератор?

```
anyList : Fuel → (Fuel → Gen0 a) → Gen0 (List a)
anyList extF1 genA = anyListImpl extF1 where
  anyListImpl : Fuel → Gen0 (List a)
  anyListImpl Dry = pure []
  anyListImpl (More f1) = oneOf [ pure []
                                , do x ← genA extF1
                                  xs ← anyListImpl f1
                                  pure (x::xs)
                                ]
```

[illegible]

Генератор индексированного типа?

Генератор индексированного типа?

```
data Fin : Nat → Type
```


Генератор индексированного типа?

```
data Fin : Nat → Type
```

```
niceFin : Fin 6
```

```
niceFin = 5
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → Gen0 (Fin n)
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → {0 n : Nat} → Gen0 (Fin n)
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → (n : Nat) → Gen0 (Fin n)
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → (n : Nat) → Gen0 (Fin n)
```

```
genAnyFin : Fuel → Gen0 (n ** Fin n)
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → (n : Nat) → Gen MaybeEmpty (Fin n)
```

```
genAnyFin : Fuel → Gen MaybeEmpty (n ** Fin n)
```

Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → (n : Nat) → Gen MaybeEmpty (Fin n)
```

```
genAnyFin : Fuel → Gen NonEmpty (n ** Fin n)
```


Генератор индексированного типа?

```
data Fin : Nat → Type
niceFin : Fin 6
niceFin = 5
failing
  badFin : Fin 6
  badFin = 6
```

```
genFin : Fuel → (n : Nat) → Gen MaybeEmpty (Fin n)
genFinS : Fuel → (n : Nat) → Gen NonEmpty (Fin (S n))

genAnyFin : Fuel → Gen NonEmpty (n ** Fin n)
```

Генератор очень индексированного типа?

Генератор очень индексированного типа?

```
data LT : Nat → Nat → Type
```

Генератор очень индексированного типа?

```
data LT : Nat → Nat → Type
```

```
genLT : Fuel → Gen NonEmpty (n ** m ** n `LT` m)
```

Генератор очень индексированного типа?

data LT : Nat → Nat → Type

genLT : Fuel → Gen NonEmpty (n ** m ** n `LT` m)

genLT' : Fuel → (n : Nat) → Gen NonEmpty (m ** n `LT` m)

Генератор очень индексированного типа?

data LT : Nat → Nat → Type

genLT : Fuel → Gen NonEmpty (n ** m ** n `LT` m)

genLT' : Fuel → (n : Nat) → Gen NonEmpty (m ** n `LT` m)

genLT_ : Fuel → (m : Nat) → Gen MaybeEmpty (n ** n `LT` m)

Генератор очень индексированного типа?

```
data LT : Nat → Nat → Type
```

```
genLT : Fuel → Gen NonEmpty (n ** m ** n `LT` m)
```

```
genLT' : Fuel → (n : Nat) → Gen NonEmpty (m ** n `LT` m)
```

```
genLT_ : Fuel → (m : Nat) → Gen MaybeEmpty (n ** n `LT` m)
```

```
genLT'_ : Fuel → (n, m : Nat) → Gen MaybeEmpty (n `LT` m)
```

В прошлых сериях
○○○○

Сигнатура генератора
○○○○●

Внутренность: обзор
○○○○

Внутренность: DepTyCheck
○○○

Заключение
○○

Запуск

Запуск

`unGen1` : `MonadRandom m` $\Rightarrow$ `Gen NonEmpty a` $\rightarrow$ `m a`

Запуск

`unGen1` : `MonadRandom m` $\Rightarrow$ `Gen NonEmpty a` $\rightarrow$ `m a`

`unGen` : `MonadRandom m` $\Rightarrow$ `Gen MaybeEmpty a` $\rightarrow$ `m (Maybe a)`

Запуск

`unGen1` : `MonadRandom m` $\Rightarrow$ `Gen NonEmpty a` $\rightarrow$ `m a`

`unGen` : `MonadRandom m` $\Rightarrow$ `Gen em a` $\rightarrow$ `m (Maybe a)`

Запуск

`unGen1` : `MonadRandom m` $\Rightarrow$ `Gen NonEmpty a` $\rightarrow$ `m a`

`unGenAll` : `RandomGen g` $\Rightarrow$ `(seed : g)` $\rightarrow$ `Gen NonEmpty a` $\rightarrow$ `Stream a`

`unGen` : `MonadRandom m` $\Rightarrow$ `Gen em a` $\rightarrow$ `m (Maybe a)`

Запуск

`unGen1 : MonadRandom m ⇒ Gen NonEmpty a → m a`

`unGenAll : RandomGen g ⇒ (seed : g) → Gen NonEmpty a → Stream a`

`unGen : MonadRandom m ⇒ Gen em a → m (Maybe a)`

`unGenTryAll : RandomGen g ⇒ g → Gen em a → Stream (Maybe a)`

Запуск

`unGen1 : MonadRandom m ⇒ Gen NonEmpty a → m a`

`unGenAll : RandomGen g ⇒ (seed : g) → Gen NonEmpty a → Stream a`

`unGen : MonadRandom m ⇒ Gen em a → m (Maybe a)`

`unGenTryAll : RandomGen g ⇒ g → Gen em a → Stream (Maybe a)`

`unGenTryN : RandomGen g ⇒ (n : Nat) → g → Gen em a → LazyList a`

Запуск

`unGen1 : MonadRandom m ⇒ Gen NonEmpty a → m a`

`unGenAll : RandomGen g ⇒ (seed : g) → Gen NonEmpty a → Stream a`

`unGen : MonadRandom m ⇒ Gen em a → m (Maybe a)`

`unGenTryAll : RandomGen g ⇒ g → Gen em a → Stream (Maybe a)`

`unGenTryN : RandomGen g ⇒ (n : Nat) → g → Gen em a → LazyList a`

`allDetermValues : Gen em a → LazyList a`

Запуск

`unGen1 : MonadRandom m  $\Rightarrow$  Gen NonEmpty a  $\rightarrow$  m a`

`unGenAll : RandomGen g  $\Rightarrow$  (seed : g)  $\rightarrow$  Gen NonEmpty a  $\rightarrow$  Stream a`

`unGen : MonadRandom m  $\Rightarrow$  Gen em a  $\rightarrow$  m (Maybe a)`

`unGenTryAll : RandomGen g  $\Rightarrow$  g  $\rightarrow$  Gen em a  $\rightarrow$  Stream (Maybe a)`

`unGenTryN : RandomGen g  $\Rightarrow$  (n : Nat)  $\rightarrow$  g  $\rightarrow$  Gen em a  $\rightarrow$  LazyList a`

`allDetermValues : Gen em a  $\rightarrow$  LazyList a`

Особенности дизайна

Особенности дизайна (без shrinking'a)

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск
Haskell	QuickCheck	IO	да, retry/ошибка

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh
Haskell	QuickCheck	I0	да, retry/ошибка	R

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	Size→Seed→a

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	Size→Seed→a
Haskell	Hedgehog	I0	да, retry/ошибка	R	Size→Seed→Tree a

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	IO	да, retry/ошибка	R	Size→Seed→a
Haskell	Hedgehog	IO	да, retry/ошибка	R	Size→Seed→Tree a
Idris	Hedgehog	чистый	нет	R	Size→Seed→Cotree a

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	IO	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	IO	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	IO	да, []	E	$\text{Series } a$

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	I0	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	I0	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a
Idris	DepTyCheck				

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	I0	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a
Idris	DepTyCheck	чистый	* ¹		

¹typelevel-контроль

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	I0	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a
Idris	DepTyCheck	чистый	* ¹ , Nothing		

¹typelevel-контроль

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	I0	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	I0	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a
Idris	DepTyCheck	чистый	* ¹ , Nothing	R/E	

¹typelevel-контроль

Особенности дизайна (без shrinking'a)

Язык	Библиотека	Запуск	Пустые, запуск	Exh	Суть генератора
Haskell	QuickCheck	IO	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow a$
Haskell	Hedgehog	IO	да, retry/ошибка	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Tree } a$
Idris	Hedgehog	чистый	нет	R	$\text{Size} \rightarrow \text{Seed} \rightarrow \text{Cotree } a$
Rocq	QuickChick	чистый	нет	R	$\text{nat} \rightarrow \text{Seed} \rightarrow A$
Haskell	*SmallCheck	IO	да, []	E	$\text{Series } a$
Haskell	Hughes' RGen	хитрый	да, Nothing	R/E	дерево значений a
Idris	DepTyCheck	чистый	* ¹ , Nothing	R/E	дерево значений a

¹typelevel-контроль

В прошлых сериях
○○○○

Сигнатура генератора
○○○○○

Внутренность: обзор
○●○○

Внутренность: DepTyCheck
○○○

Заключение
○○

Дерево значений?

Дерево значений?

- John Hughes' RGen

- DepTyCheck

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024
- DepTyCheck

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024*
- DepTyCheck

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
- DepTyCheck

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
- DepTyCheck
 - Опубликовано в 2021

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
- DepTyCheck
 - Опубликовано в 2021

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
 - Крайне быстрый в некоторых ситуациях фильтрации
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
 - Крайне быстрый в некоторых ситуациях фильтрации
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке
 - Сравним по скорости в некоторых ситуациях

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
 - Крайне быстрый в некоторых ситуациях фильтрации
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке
 - Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
 - Крайне быстрый в некоторых ситуациях фильтрации
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке
 - Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри

Дерево значений?

- John Hughes' RGen

- Опубликовано в 2024**
- На Haskell, ленивом и практически чистом языке
- Крайне быстрый в некоторых ситуациях фильтрации
- Практически нечувствителен к порядку фильтрации

- DepTyCheck

- Опубликовано в 2021
- На Idris, неленивом (lazy on demand) и чистом языке
- Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри

Дерево значений?

- John Hughes' RGen

- Опубликовано в 2024**
- На Haskell, ленивом и практически чистом языке
- Крайне быстрый в некоторых ситуациях фильтрации
- Практически нечувствителен к порядку фильтрации

- DepTyCheck

- Опубликовано в 2021
- На Idris, неленивом (lazy on demand) и чистом языке
- Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри
- Крайне чувствителен в порядку фильтрации

Дерево значений?

- John Hughes' RGen

- Опубликовано в 2024**
- На Haskell, ленивом и практически чистом языке
- Крайне быстрый в некоторых ситуациях фильтрации
- Практически нечувствителен к порядку фильтрации
 - Существенно использует ленивость и механизм exception'ов Haskell

- DepTyCheck

- Опубликовано в 2021
- На Idris, неленивом (lazy on demand) и чистом языке
- Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри
- Крайне чувствителен в порядку фильтрации

Дерево значений?

- John Hughes' RGen

- Опубликовано в 2024**
- На Haskell, ленивом и практически чистом языке
- Крайне быстрый в некоторых ситуациях фильтрации
- Практически нечувствителен к порядку фильтрации
 - Существенно использует ленивость и механизм exception'ов Haskell

- DepTyCheck

- Опубликовано в 2021
- На Idris, неленивом (lazy on demand) и чистом языке
- Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри
- Крайне чувствителен к порядку фильтрации
 - Подбирает оптимальный при деривации

Дерево значений?

- John Hughes' RGen

- Опубликовано в 2024**
- На Haskell, ленивом и практически чистом языке
- Крайне быстрый в некоторых ситуациях фильтрации
- Практически нечувствителен к порядку фильтрации
 - Существенно использует ленивость и механизм exception'ов Haskell
- Крайне чувствителен к типу проверок

- DepTyCheck

- Опубликовано в 2021
- На Idris, неленивом (lazy on demand) и чистом языке
- Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри
- Крайне чувствителен к порядку фильтрации
 - Подбирает оптимальный при деривации

Дерево значений?

- John Hughes' RGen
 - Опубликовано в 2024**
 - На Haskell, ленивом и практически чистом языке
 - Крайне быстрый в некоторых ситуациях фильтрации
 - Практически нечувствителен к порядку фильтрации
 - Существенно использует ленивость и механизм exception'ов Haskell
 - Крайне чувствителен к типу проверок
- DepTyCheck
 - Опубликовано в 2021
 - На Idris, неленивом (lazy on demand) и чистом языке
 - Сравним по скорости в некоторых ситуациях
 - Распределяет фильтрацию между созданием и запуском
 - Существенно использует зависимые типы внутри
 - Крайне чувствителен к порядку фильтрации
 - Подбирает оптимальный при деривации
 - Нечувствителен к типу проверок

Дерево значений

```
data RGen a = Fail
           | Value a
           | Random Rational (RGen a) (RGen a)

(>>=) :: RGen a -> (a -> RGen b) -> RGen b

a >>= _ | isFail a                = Fail
_ >>= h | isFail (h (throw Undef)) = Fail
Value x >>= h                    = h x
Random p a b >>= h               = Random p (a >>= h)
                                   (b >>= h)

isFail x = unsafePerformIO $
           case x of Fail -> return True
                  _    -> return False
           `catch` \Undef -> return False
```

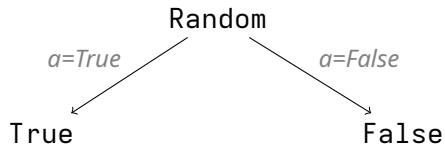


Дерево значений

```
gen :: RGen Bool
gen = do
  a ← elements [True, False]
  pure a
```

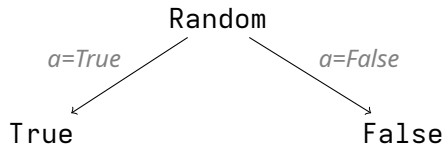
Дерево значений

```
gen :: RGen Bool  
gen = do  
  a ← elements [True, False]  
  pure a
```



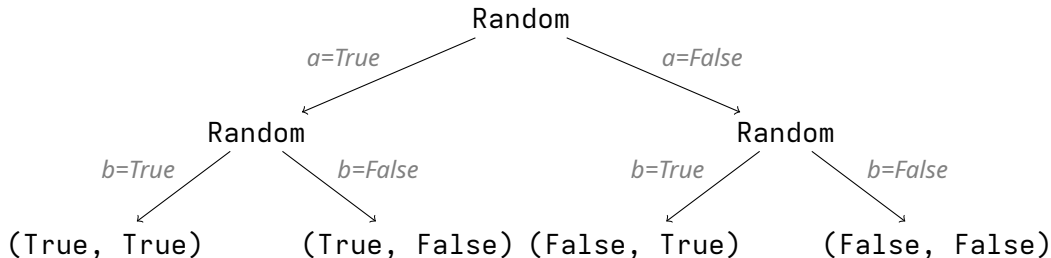
Дерево значений

```
gen :: RGen (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  pure (a, b)
```



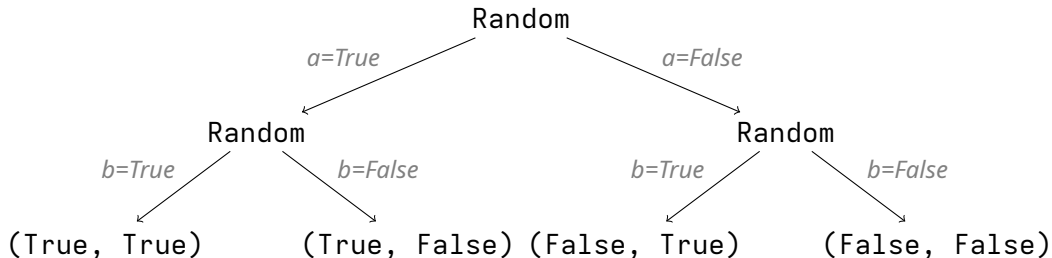
Дерево значений

```
gen :: RGen (Bool, Bool)
gen = do
  a <- elements [True, False]
  b <- elements [True, False]
  pure (a, b)
```



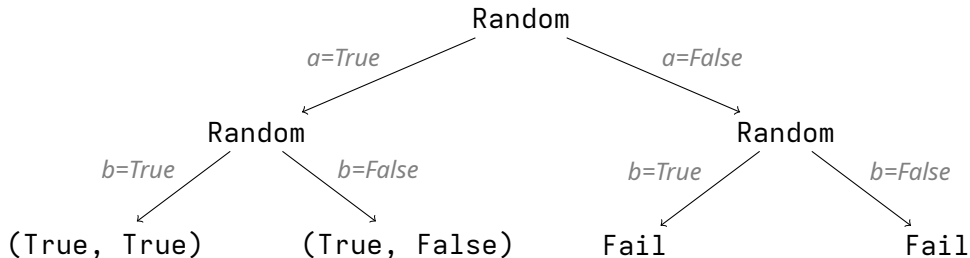
Дерево значений

```
gen :: RGen (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  guard a
  pure (a, b)
```



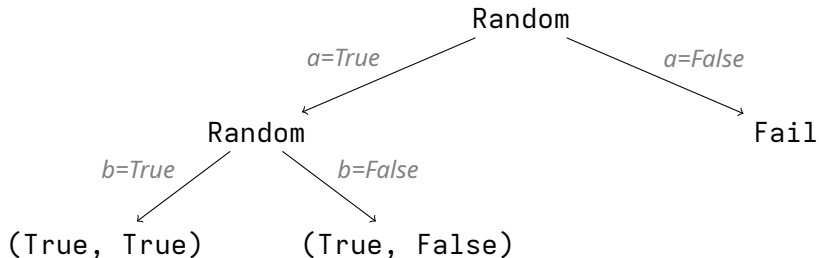
Дерево значений

```
gen :: RGen (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  guard a
  pure (a, b)
```



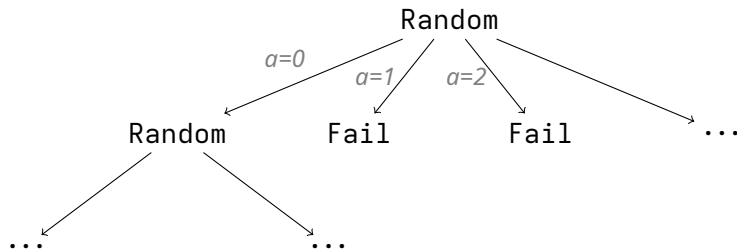
Дерево значений

```
gen :: RGen (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  guard a
  pure (a, b)
```



Дерево значений

```
gen :: RGen (Int, Int)
gen = do
  a ← elements [0..100]
  b ← elements [0..100000]
  guard (a `mod` 20 == 0)
  pure (a, b)
```



¹Я упрощаю дерево, оно же, на самом деле, бинарное

В прошлых сериях
○○○○

Сигнатура генератора
○○○○○

Внутренность: обзор
○○○○

Внутренность: DepTyCheck
●○○

Заключение
○○

А у DepTyCheck'a?

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```


А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where  
  Empty : Gen MaybeEmpty a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
data Gen : Emptiness → Type → Type where
  Empty  : Gen MaybeEmpty a
  Pure   : a → Gen em a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where
```

```
  Empty : Gen MaybeEmpty a
```

```
  Pure   : a → Gen em a
```

```
  OneOf  : LazyList (Nat, Lazy (Gen em a)) → Gen em a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where
```

```
  Empty : Gen MaybeEmpty a
```

```
  Pure   : a → Gen em a
```

```
  OneOf  : LazyList1 (Nat, Lazy (Gen em a)) → Gen em a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where
```

```
  Empty : Gen MaybeEmpty a
```

```
  Pure   : a → Gen em a
```

```
  OneOf  : LazyList1 (Nat, Lazy (Gen NonEmpty a)) → Gen em a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
```

```
data Gen : Emptiness → Type → Type where
```

```
  Empty : Gen MaybeEmpty a
```

```
  Pure  : a → Gen em a
```

```
  OneOf : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
```

А у DepTyCheck'a?

- Из практических соображений, небинарные деревья
- Нужны ранние сокращения
- Нет трюка с `isFail (h (throw Undef))`
- Но есть зависимые типы!

```
data Emptiness = NonEmpty | MaybeEmpty
data Gen : Emptiness → Type → Type where
  Empty  : Gen MaybeEmpty a
  Pure   : a → Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
```

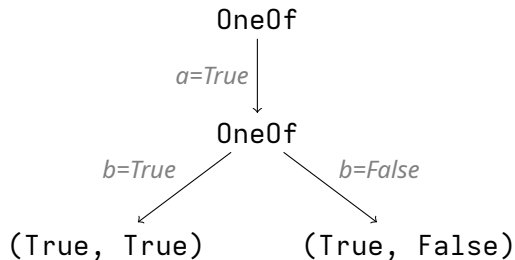
¹Но, конечно же, всё сложнее...

A y DepTyCheck'a

```
gen : Gen MaybeEmpty (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  guard a
  pure (a, b)
```

A y DepTyCheck'a

```
gen : Gen MaybeEmpty (Bool, Bool)
gen = do
  a ← elements [True, False]
  b ← elements [True, False]
  guard a
  pure (a, b)
```

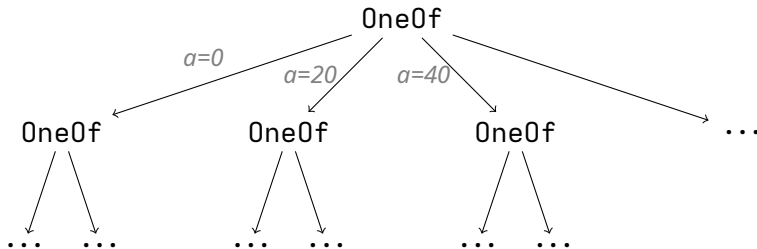


A y DepTyCheck'a

```
gen : Gen MaybeEmpty (Int, Int)
gen = do
  a ← elements [0..100]
  b ← elements [0..100000]
  guard (a `mod` 20 == 0)
  pure (a, b)
```

A y DepTyCheck'a

```
gen : Gen MaybeEmpty (Int, Int)
gen = do
  a ← elements [0..100]
  b ← elements [0..100000]
  guard (a `mod` 20 == 0)
  pure (a, b)
```



Конечно, всё сложнее

```
data Gen : Emptiness → Type → Type where
  Empty : Gen MaybeEmpty a
  Pure   : a → Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
```

Конечно, всё сложнее

- Нужна поддержка меток

```
data Gen : Emptiness → Type → Type where
  Empty : Gen MaybeEmpty a
  Pure   : a → Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
```

Конечно, всё сложнее

- Нужна поддержка меток

```
data Gen : Emptiness → Type → Type where
  Empty : Gen MaybeEmpty a
  Pure   : a → Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
  Labelled : Label → Gen NonEmpty a → Gen em a
```

Конечно, всё сложнее

- Нужна поддержка меток
- Нужна поддержка “сырых” генераторов

```
data Gen : Emptiness → Type → Type where
  Empty  : Gen MaybeEmpty a
  Pure   : a → Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) → Gen em a
  Labelled : Label → Gen NonEmpty a → Gen em a
```


Конечно, всё сложнее

- Нужна поддержка меток
- Нужна поддержка “сырых” генераторов

```
record RawGen a where
  constructor MkRawGen
  unRawGen : forall m. MonadRandom m => m a

data Gen : Emptiness -> Type -> Type where
  Empty : Gen MaybeEmpty a
  Pure   : a -> Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) -> Gen em a
  Labelled : Label -> Gen NonEmpty a -> Gen em a
```

Конечно, всё сложнее

- Нужна поддержка меток
- Нужна поддержка “сырых” генераторов

```
record RawGen a where
  constructor MkRawGen
  unRawGen : forall m. MonadRandom m => m a

data Gen : Emptiness -> Type -> Type where
  Empty : Gen MaybeEmpty a
  Pure   : a -> Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) -> Gen em a
  Labelled : Label -> Gen NonEmpty a -> Gen em a
  Raw     : RawGen a -> Gen em a
```

Конечно, всё сложнее

- Нужна поддержка меток
- Нужна поддержка “сырых” генераторов

```
record RawGen a where
  constructor MkRawGen
  unRawGen : forall m. MonadRandom m => m a

data Gen : Emptiness -> Type -> Type where
  Empty : Gen MaybeEmpty a
  Pure   : a -> Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) -> Gen em a
  Labelled : Label -> Gen NonEmpty a -> Gen em a
  Raw     : RawGen a -> Gen em a
  Bind    : RawGen a -> (a -> Gen em b) -> Gen em b
```

Конечно, всё сложнее

- Нужна поддержка меток
- Нужна поддержка “сырых” генераторов

```
record RawGen a where
  constructor MkRawGen
  unRawGen : forall m. MonadRandom m => m a

data Gen : Emptiness -> Type -> Type where
  Empty : Gen MaybeEmpty a
  Pure   : a -> Gen em a
  OneOf  : LazyList1 (Nat1, Lazy (Gen NonEmpty a)) -> Gen em a
  Labelled : Label -> Gen NonEmpty a -> Gen em a
  Raw     : RawGen a -> Gen em a
  Bind    : RawGen a -> (a -> Gen em b) -> Gen em b
```

¹Но, конечно же, всё ещё сложнее...

В прошлых сериях
○○○

Сигнатура генератора
○○○○

Внутренность: обзор
○○○

Внутренность: DepTyCheck
○○○

Заключение
●○

Напоследок

Спасибо!

Вопросы?